

Modeling reciprocal adaptation in HCI: a Multi-Agent Reinforcement Learning Approach

Tanay Gupta

f20180343@hyderabad.bits-pilani.ac.in
BITS Pilani
Hyderabad, India

Julien Gori

gori@isir.upmc.fr
ISIR, CNRS, Sorbonne Université
Paris, France

ABSTRACT

Adaptation between users and computers is difficult because of the reciprocal long-term adaptation between the user and an adaptive tool. In this work in progress, we present a novel method for designing adaptive systems, by simulating reciprocal adaptation between a user and an intelligent tool via Multi-Agent Reinforcement Learning (MARL). We report on a simple target-selection simulation in which a simulated, rational user tries to reach a target and is aided by an intelligent tool that manipulates the user's environment. The user and the tool's behavior are jointly determined by an adaptive algorithm that maximizes their utility. Our approach shows that both agents are capable of learning realistic policies, with the user and tool combination being 30% more effective than the user alone.

CCS CONCEPTS

• Human-centered computing → HCI theory, concepts and models.

KEYWORDS

co-evolution, human-computer interaction, deep reinforcement learning, multi-agent systems

ACM Reference Format:

Tanay Gupta and Julien Gori. 2023. Modeling reciprocal adaptation in HCI: a Multi-Agent Reinforcement Learning Approach. In *Extended Abstracts of the 2023 CHI Conference on Human Factors in Computing Systems (CHI EA '23)*, April 23–28, 2023, Hamburg, Germany. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3544549.3585913>

1 INTRODUCTION

Users of computing systems come with varying abilities and preferences. Designing a single interface for a wide array of users is thus a complex problem. *Adaptive* interfaces are systems whose behavior purposely depends on the particular user they are interacting with. This is usually achieved by a combination of sensing the user's psychophysical states and mechanisms of decision-making that depend on the system's environment, building on inference and prediction [19]. Adaptive systems constitute an essential lever to enhance the accessibility and user-efficiency of computing systems,

which lies at the intersection between human-computer interaction (HCI) and Artificial Intelligence (AI).

Despite many efforts (see Section 2), designing adaptive interfaces remains a challenging problem [32]. One difficulty is the *reciprocal adaptation* between the user of an adaptive system and the system itself. Indeed, the adaptive system is sensitive to the user's behavior by design. At the same time, however, the user is also learning, thus generating novel behavior [22]. This reciprocal adaptation may unfold over a long period of time, which makes empirical evaluations of adaptive systems very challenging [39]; design methods that rely heavily on iterations between design phases and empirical evaluations (e.g., user-centered design), are thus hard to conduct in this context.

In this work in progress, we make the hypothesis that the design of adaptive systems can be bootstrapped by relying on computational (simulated) user models [29] to reduce the reliance on empirical evaluations. In that perspective, the quality of user models is primordial to achieve decent designs; in particular, these models should display reciprocal adaptation *i.e.*, user and agent actions should be jointly determined. We report preliminary results on a toy pointing facilitation technique, where a user has to select a target, aided by an intelligent cursor. The user and cursor's behavior are *jointly* determined by a Multi-Agent Reinforcement Learning (MARL) algorithm that minimizes the number of user actions needed to reach the target. The learned user behavior is plausible and, taken with the learned cursor behavior, selection time with assistance is 30% more efficient than without assistance. Overall, this work shows the potential of using MARL algorithms to simulate adaptive systems, with the final goal being to aid design.¹

2 BACKGROUND

Human Computer Collaboration. The idea of humans collaborating with computers is not new, we refer to Fischer [9] and Terveen [34] for an overview that is still relevant today. Closest to the present work are mixed-initiative systems (MIS): collaborative systems where both the user and the system have agency. These are well described in the literature [15, 16] and high-level principles to guide the design of MIS exist (e.g., Horvitz [15] lists 12). Generic questions such as “when should the MIS take a particular action?” [15] or “when should the MIS solicit input from the user?” [10] can be answered, but usually involve computations that require quantities that are hard to determine (e.g., the probability that a user could be made to understand an intervention, or the user-perceived cost of a particular intervention).

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CHI EA '23, April 23–28, 2023, Hamburg, Germany

© 2023 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9422-2/23/04.

<https://doi.org/10.1145/3544549.3585913>

¹The code for this work is available at <https://github.com/Tanay0101/AI-Input-Design>

Adaptive interfaces. Adaptive systems are systems that conform to the user's needs, making them a key mechanism to cope with the diversity of users [27]. Several design methods for adaptive interfaces exist; a very popular and —from the standpoint of computational simulations— a convenient method, is to formulate the design problem as one of optimization, where a cost that correlates with user needs is minimized. This idea is already quite old in HCI, dating at least from the nineties [26] if not before, and still relevant today. For example, Todi *et al.* [35] designed an adaptive menu that minimizes the average menu item selection time. Similarly, Gajos *et al.* [11] designed an adaptive interface's layout to reduce navigation time. In this work, the assistive cursor minimizes the expected target selection time.

Human adaptation. Our nervous system adapts rapidly to changes in its environment, even though the actual mechanisms behind adaptation are still not well understood [2]. While initially adaptation was conceived as correcting behavior towards a baseline [38], recent accounts suggest that adaptation emerges from the human tendency to continuously search for the best behavior [18]. The core assumption, which we also make in this work, is that human behavior is (boundedly) optimal [12, 17], and is obtained by maximizing an internal utility function [13, 36]. In this view, adaptive behavior thus emerges because the user's best strategy has shifted due to changes in the environment [7, 21]. In this work, we consider that the user attempts to minimize its expected target selection time.

In summary: in this work, we view human adaptation as the perpetual optimization of behavior in response to a changing environment; the design of adaptive systems can be formulated as an optimization problem. As a result, reciprocal adaptation can be described by a *joint optimization* of system and user behavior. In the context of sequential interactions, common in HCI, MARL thus emerges as a promising method, which this work starts investigating.

MARL. Reinforcement learning (RL) is used to solve sequential decision-making problems. In single-agent RL, an agent interacts with the environment, and at each time step t , it observes the environment's state s_t , and takes an action a_t . How the agent selects an action is defined by the agent's *policy* $\pi(a_t|s_t)$, a (probabilistic) mapping from state s_t to action a_t . If the observation is partial *i.e.*, the state is not perfectly observed, an observation function $O(s)$ produces observations o_t from states s_t . The policy $\pi(a_t|o_t)$ is then based on observations rather than states. The environment then moves to a new state s_{t+1} and the agent receives a reward R_{t+1} [33]. The goal in RL is to find the policy that maximizes the total (discounted) reward obtained during an episode (*i.e.*, when completing a task).

Multi-Agent RL (MARL) generalizes the RL setting to the case of multiple agents [41]. The multi-agent setting is challenging because from the point of view of any agent, the environment is non-stationary and information is incomplete [8, 25]. For example, in our scenario, the goal target is unknown to the cursor.²

Several algorithms have been developed for the MARL setting [4, 30]; in particular; Lowe *et al.* [25] derived a Multi-Agent Deep

Deterministic Policy Gradient (MADDPG) algorithm that builds on Centralized Training with Decentralized Execution (CTDE). In CTDE, all information is available during training (as if the agents could share the missing information with each other) but becomes incomplete during the actual execution of the agent's policy (as it should be). Our work builds on the existing MADDPG algorithm³.

3 PROTOTYPICAL POINTING FACILITATION SETUP

3.1 The Target-selection Task

We consider the target-selection task represented Fig. 1:

- A grid (N) is represented as a binary matrix, where cells with icons are represented with ones and others are represented with zeros. One of the icons is the user's goal (green) with coordinates G , and a physical cursor (red) with coordinates C . The task is completed when the red cursor falls on the green target, or after the number of steps exceeds the maximum allowed steps. The complete state of the system can be defined as $s(t) = (N, G, C)$.
- User actions α_u are the 4 cardinal directions (*up*, *down*, *left* and *right*).
- The assistant selects a gain $\alpha_a \in [1, 4]$ to modulate the user actions.
- The cursor is moved as: $C_t = C_{t-1} + \alpha_u \times \alpha_a$. For example if the assistant selects a gain $\alpha_a = 2$ and the user has selected $\alpha_u = up$, the cursor will move two units up.
- The user observation $O_u(s) = \{G, C\}$ consists of current cursor location C and user's goal coordinates G .
- The user policy $\pi_u(\alpha_u|O_u)$ maps user observations O_u to user action (α_u).
- The assistant observation is $O_a(s) = \{N, C_{t-m+1}, \dots, C_t, \alpha_{u_{t-m+1}}, \dots, \alpha_{u_t}\}$, where m is the memory length, $(C_{t-m+1}, \dots, C_t)$ and $(\alpha_{u_{t-m+1}}, \dots, \alpha_{u_t})$ are the previous cursor locations and user actions, that have been one-hot encoded. To summarize, the goal G is hidden to the assistant, but the assistant can see the remaining information (including past m cursor locations and user actions).
- The assistant policy $\pi_a(\alpha_a|O_a)$ maps O_a to assistant action (α_a).
- Both agents are rewarded with -1 at each step and $+10$ if they reach the target icon. User and assistant policies that maximize the return will thus cooperate to locate the target with as few steps as possible.

This problem can be modeled as a Markov Game [23] since neither the user nor the agent has complete knowledge of the state of the system (the user does not know the gain that will be applied to its movement and the assistant does not know the target icon) and state transitions are assumed Markovian. To solve this problem, we need to determine the policies $\pi_u(\alpha_u|O_u)$ and $\pi_a(\alpha_a|O_a)$ of both agents, and we are going to do so using MARL. From an HCI perspective, the task models two cases of input facilitation: One with a user of a keypad, keyboard arrows, or an input device

²There is also a so-called credit assignment issue, where the agent can not determine if his action or another agent's lead to the reward, but we don't have that problem here.

³but using a simplified version with a single critic, whereas MADDPG has one critic per agent.

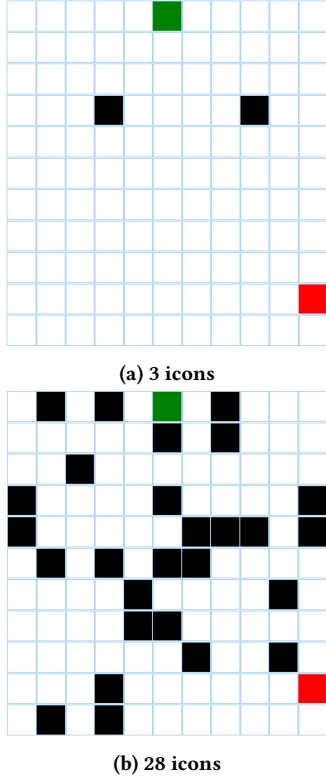


Figure 1: Icon layout with red icon denoting the target icon and green cell denoting user location.

with similarly low expressivity. For example, brain signals can be classified into two or four classes (up, down, left, right) to move a cursor in one and two dimensions [37, 40]. The second case is as a preliminary step towards dealing with the complete computer mouse input, by considering only direction and not amplitude.

3.2 Target-selection Facilitation as a Multi Agent Decision-making Problem

Existing target-selection facilitation tools from the HCI literature can be recovered for various cursor policies:

Constant CD Gain is recovered with the constant policy $\pi(\alpha_a|s) = k_0$, which always applies the same gain k_0 [5, 6] to user input.

Non-linear CD Gain is recovered with the policy $\pi(\alpha_a|s) = f(\alpha_u)$, where f is usually an arbitrary deterministic mapping. This is the most common pointing facilitation technique, used on most mainstream OS's [5, 6].

Target Aware techniques Many researchers have tried to take advantage of the extra information that is contained in the target locations $\{T_i\}_{i=1}^n$ to enhance target-selection, by having policies of the type $\pi(\alpha_a|s) = g(\{T_i\}_{i=1}^n, \alpha_u)$, where the gain is picked based on the potential targets and user actions. Object pointing [14] and semantic pointing [3] are two well known examples (see [20, Table 1] for more examples).

General case In the MARL setting, the assistant policy is able to make use of all the information that is available, and performance is theoretically better (more information can't be worse), even though it may practically be hard to attain. Extra sensing techniques such as eye-tracking or providing more target-relevant information, such as a belief regarding which target is most likely to be the user's goal could even be added. This last idea is used for example in BIGPoint [24].

4 METHODS

4.1 Two-Agent PPO-based Algorithm

We devised a two-agent PPO-based algorithm, by adapting the MADDPG algorithm introduced in [25]. We reused their multi-agent actor-critic architecture with a centralized action-value function (critic), but replaced the DDPG component with the PPO component as described in [31]. In contrast to Lowe *et al.* however, we use a simplified architecture with a single critic, whereas Lowe *et al.* used one critic per agent. This is because, in the present case of user assistance, we assume that the assistant's goal is perfectly aligned with that of the user's. The primary motivation behind using a centralized action-value function is that, if we know the actions taken by all agents, the environment becomes stationary even if the agent policies change over time, which is one of the difficulties of MARL. Following CTDE, the networks were trained using the complete information about the state and actions of the agents, but incomplete state information was used during evaluation. In particular, the user goal is not known to the assistive agent at time of execution, as it should realistically be, but is available during training.

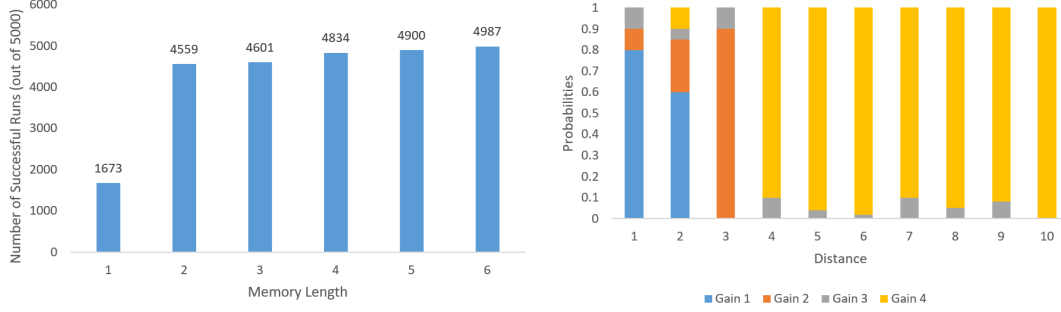
The architectures used for the user, assistant and critic networks respectively (Fig. 4), and the update equations for the assistant, user and critic networks can be found in the Appendix.

4.2 Simulation & Training

To avoid overfitting to a particular icon layout, the number of icons in the layout was uniformly sampled from [3, 30], and their locations were randomized, in every iteration. Fig. 1 shows two possible grid examples. The start location is chosen randomly as well for every iteration, while the target location is randomly selected from the available icons. We used curriculum learning [1] to ease training of the agents: We allowed the user to take a maximum of 40 steps during the start of training, but this maximum number of steps was then slowly constrained to 6 to motivate the assistant to exploit the information available and reach the target more effectively since we noticed that otherwise the agents got stuck in local optima and failed to learn. We did not constrain the steps beyond six as this makes certain grid regions inaccessible despite optimal play on the part of the assistant, i.e. the user will not be able to reach from one corner of the grid to the opposite corner in less than six steps. During training, user and assistant actions are sampled based on the probability of each output, but during execution, the actions selected are those that correspond to the output with the highest probability. This is a standard practice used in reinforcement learning to remove the effect of exploration during the final evaluation of the policies. Table 1 lists the values of hyperparameters used.

Table 1: Hyperparameter configurations for the experiment

Hyperparameter	Description	Value
λ_C	Learning rate of critic networks	0.002
λ_U	Learning rate of user network	0.001
λ_A	Learning rate of assistant network	0.001
γ	Discount factor	0.95
Steps _{max}	Maximum steps (at start)	40
Steps _{min}	Minimum steps (after constraint)	6
$\epsilon_{\text{constraint}}$	Rate of constraining	2 steps per 100 epochs
Memory Size	Information about previous steps + current user step	6

**Figure 2: Left: Effect of Memory Length on Successful Runs. Right: Effect of Distance on Probability of Gain**

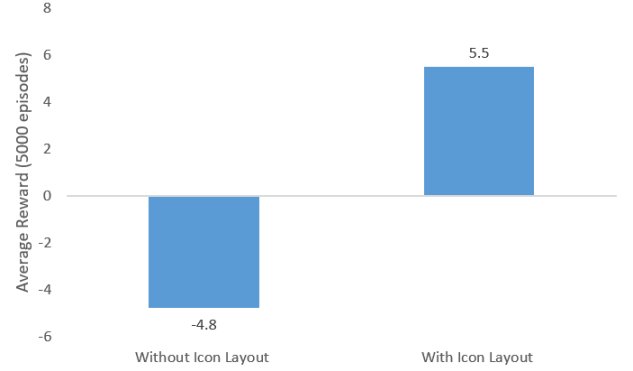
5 RESULTS

We first verified that both agents were able to learn. To do so, we introduced a single-agent baseline where a user is coupled with an assistant that provides constant unit gain. In that case, the number of steps required by the optimal user can be expressed explicitly as the Manhattan distance between the start and target coordinates, given by $|G_x - C_x| + |G_y - C_y|$. After joint learning of user and assistant policies, we observed a reduction of 30% in the required number of steps in comparison with the baseline, which indicates our agents did learn.

5.1 Effect of Memory Length

Memory length is an important hyperparameter as it provides the assistant information about previous steps. For example, if the assistant overshoots the target and the user tries to return to it, the assistant must exploit past information to avoid overshooting it again: memory helps the assistant make an educated guess about the target location.

We conducted experiments to check the effect of memory length, by varying the memory size of the assistant’s network. Fig 2 shows the relation between memory length and successful runs: increasing memory length from 1 to 2 leads to a major improvement, which highlights the importance of using a network with memory for the assistant. We suppose this is because it allows the assistant to determine which target is the user’s goal, based on past user actions. We used Pearson’s chi-squared test to analyze the relationship between the increase in memory length and successful task completion. Transition in memory size from 1 to 2 and from 3 to 4 were significant at $\alpha = 0.05$ ($p_{1,2} < 0.0001$, $p_{2,3} = 0.661$, $p_{3,4} = 0.017$, $p_{4,5} = 0.504$, $p_{5,6} = 0.382$). This suggests that, in this particular task, the last two user actions contain most of the information needed by the assistive agent to detect the correct target.

**Figure 3: Effect of Icon Layout Information on Final Reward**

5.2 Effect of Icon Layout Information

Here, we performed an ablation study, where we removed the grid information (N) from the state. The assistant fails to find a good policy to reach the target icon when the icon layout information is not provided, as shown by the right panel of Fig. 3. During evaluation, we limited the number of actions for each agent to 6 (the minimum possible reward is thus -6). Without the layout, information agents reach a reward of -4.8 after training. In contrast, when the assistant is provided the layout information, agents have an average reward of 5.5 when training ends, which shows learning from the grid to be effective. This suggests the assistive agent is able to exploit target layout information.

5.3 Relation between Gain selected by Assistant and Distance from Target

We measured the probability of the assistant selecting a gain as a function of its distance from the target icon. With perfect observations and optimal policy, the gain should be equal to the remaining distance when the distance is less or equal to 4 steps, and otherwise 4. However, with imperfect observations, the optimal policy is not known. The right panel of Fig 2 shows the relation between the distance from the target and the probability of the agent selecting the gain. Overall, the selected gains match reasonably well with the fully informed optimal baseline, although we note a conservative approach for distances 2 and 3, where respectively gains of 1 and 2 are more often selected. It is not clear whether this behavior to undershoot the target is optimal (it would be more costly to overshoot, and thus there is a bias towards undershooting), if it is due to imperfect observations (reaching a certain position *e.g.*, in the middle of a few targets may be more informative to the assistant, as in BIGPoint [24]), or if we simply found a suboptimal policy.

6 CONCLUDING REMARKS AND PERSPECTIVES

Designing and constructing an adaptive user interface is difficult [27], one of the reasons being the reciprocal adaptation between users and adaptive interfaces. In this preliminary work, we explored the feasibility of a MARL approach to modeling reciprocal adaptation in a simple target-selection problem. Results are encouraging, and show it is possible to jointly learn good policies of a user and an assistant using a centralized training decentralized execution paradigm; they also pave the way to a target facilitation human-computer partnership, where the user benefits from computational power while remaining in control of the interaction. The MARL approach is also interesting because it requires explicit acknowledgment that the human may adapt its behavior as it uses the tool. While this work suggests that designing such co-adaptive tools, that adapt to the user and to which the user adapts is possible, we are also cautious about the potential repercussions it may have on how we view interaction. Is it possible that users will end up adapting to the tool, even being encouraged to do so, rather than the tool adapting to users as intended?

In the future, we plan on extending this simple problem to a real-world target selection task, namely by investigating a large-scale setup with continuous states and action spaces. This will leave more room for the agents (in particular the user agent) to generate novel behavior. We also plan on comparing the trained user policy with policies sampled from real users, as well as comparing the designed assistants with existing target facilitation techniques, and evaluating their performance against a population of real users. In our work, the entire grid is passed to a convolutional network, which means that in a large-scale setup an image of the screen could be passed to the network. Such learning from pixels on the entire screen, which would be required for a real-world implementation of target facilitation, has been successfully demonstrated before, notably with Atari games [28], but not in the context of interaction techniques.

REFERENCES

- [1] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. Curriculum learning. In *Proceedings of the 26th annual international conference on machine learning*. 41–48.
- [2] Tao Bian, Daniel M Wolpert, and Zhong-Ping Jiang. 2020. Model-free robust optimal feedback mechanisms of biological motor control. *Neural computation* 32, 3 (2020), 562–595.
- [3] Renaud Blanch, Yves Guiard, and Michel Beaudouin-Lafon. 2004. Semantic pointing: improving target acquisition with control-display ratio adaptation. In *Proceedings of the SIGCHI conference on Human factors in computing systems*. 519–526.
- [4] Lucian Busoniu, Robert Babuska, and Bart De Schutter. 2008. A comprehensive survey of multiagent reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)* 38, 2 (2008), 156–172.
- [5] G ry Casiez and Nicolas Roussel. 2011. No more bricolage! Methods and tools to characterize, replicate and compare pointing transfer functions. In *Proceedings of the 24th annual ACM symposium on User interface software and technology*. 603–614.
- [6] G ry Casiez, Daniel Vogel, Ravin Balakrishnan, and Andy Cockburn. 2008. The impact of control-display gain on user performance in pointing tasks. *Human-computer interaction* 23, 3 (2008), 215–250.
- [7] Xiuli Chen, Aditya Acharya, Antti Oulasvirta, and Andrew Howes. 2021. An Adaptive Model of Gaze-based Selection. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. In press.
- [8] Quentin Debar, Gilles Steeve Dibangoye, Stephane Canu, and Christian Wolf. 2019. Learning 3d navigation protocols on touch interfaces with cooperative multi-agent reinforcement learning. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 35–52.
- [9] Gerhard Fischer. 2001. User modeling in human-computer interaction. *User modeling and user-adapted interaction* 11, 1 (2001), 65–86.
- [10] Michael Fleming and Robin Cohen. 2001. A user modeling approach to determining system initiative in mixed-initiative ai systems. In *International Conference on User Modeling*. Springer, 54–63.
- [11] Krzysztof Gajos and Daniel S Weld. 2004. SUPPLE: automatically generating user interfaces. In *Proceedings of the 9th international conference on Intelligent user interfaces*. 93–100.
- [12] Samuel J Gershman, Eric J Horvitz, and Joshua B Tenenbaum. 2015. Computational rationality: A converging paradigm for intelligence in brains, minds, and machines. *Science* 349, 6245 (2015), 273–278.
- [13] Julien Gori and Olivier Rioul. 2020. A Feedback Information-Theoretic Transmission Scheme (FITTS) for modeling trajectory variability in aimed movements. *Biological Cybernetics* 114, 6 (2020), 621–641.
- [14] Yves Guiard, Renaud Blanch, and Michel Beaudouin-Lafon. 2004. Object pointing: a complement to bitmap pointing in GUIs. In *Proceedings of Graphics Interface 2004*. Citeseer, 9–16.
- [15] Eric Horvitz. 1999. Principles of mixed-initiative user interfaces. In *Proceedings of the SIGCHI conference on Human Factors in Computing Systems*. 159–166.
- [16] Eric J Horvitz. 2007. Reflections on challenges and promises of mixed-initiative interaction. *AI Magazine* 28, 2 (2007), 3–3.
- [17] Andrew Howes, Richard L Lewis, and Alonso Vera. 2009. Rational adaptation under task and processing constraints: implications for testing theories of cognition and action. *Psychological review* 116, 4 (2009), 717.
- [18] Jun Izawa, Tushar Rane, Opher Donchin, and Reza Shadmehr. 2008. Motor adaptation as a process of reoptimization. *Journal of Neuroscience* 28, 11 (2008), 2883–2891.
- [19] Giulio Jacucci, Anna Spagnoli, Jonathan Freeman, and Luciano Gamberini. 2015. Symbiotic interaction: a critical definition and comparison to other human-computer paradigms. In *International workshop on symbiotic interaction*. Springer, 3–20.
- [20] Alex Jansen, Leah Findlater, and Jacob O Wobbrock. 2011. From the lab to the world: Lessons from extending a pointing technique for real-world use. In *CHI'11 Extended Abstracts on Human Factors in Computing Systems*. 1867–1872.
- [21] Yu Jiang and Zhong-Ping Jiang. 2014. Adaptive dynamic programming as a theory of sensorimotor control. *Biological cybernetics* 108, 4 (2014), 459–473.
- [22] Gordon Kurtenbach and William Buxton. 1993. The limits of expert performance using hierarchic marking menus. In *Proceedings of the INTERACT'93 and CHI'93 conference on Human factors in computing systems*. 482–487.
- [23] Michael L Littman. 1994. Markov games as a framework for multi-agent reinforcement learning. In *Machine learning proceedings 1994*. Elsevier, 157–163.
- [24] Wanyu Liu, Rafael Lucas d'Oliveira, Michel Beaudouin-Lafon, and Olivier Rioul. 2017. Bignav: Bayesian information gain for guiding multiscale navigation. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. 5869–5880.
- [25] Ryan Lowe, Yi I Wu, Aviv Tamar, Jean Harb, OpenAI Pieter Abbeel, and Igor Mordatch. 2017. Multi-agent actor-critic for mixed cooperative-competitive environments. *Advances in neural information processing systems* 30 (2017).

- [26] Pattie Maes and Robyn Kozierok. 1993. Learning interface agents. In *AAAI*, Vol. 93. 459–465.
- [27] Mahdi H Miraz, Maaruf Ali, and Peter S Excell. 2021. Adaptive user interfaces and universal usability through plasticity of user interface design. *Computer Science Review* 40 (2021), 100363.
- [28] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. 2013. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602* (2013).
- [29] Antti Oulasvirta. 2018. *Computational interaction*. Oxford University Press.
- [30] Liviu Panait and Sean Luke. 2005. Cooperative multi-agent learning: The state of the art. *Autonomous agents and multi-agent systems* 11, 3 (2005), 387–434.
- [31] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- [32] Constantine Stephanidis, Gavriel Salvendy, Margherita Antona, Jessie YC Chen, Jianming Dong, Vincent G Duffy, Xiaowen Fang, Cali Fidopiastis, Gino Fragomeni, Limin Paul Fu, et al. 2019. Seven HCI grand challenges. *International Journal of Human-Computer Interaction* 35, 14 (2019), 1229–1269.
- [33] Richard S Sutton and Andrew G Barto. 2018. *Reinforcement learning: An introduction*. MIT press.
- [34] Loren G Terveen. 1995. Overview of human-computer collaboration. *Knowledge-Based Systems* 8, 2-3 (1995), 67–81.
- [35] Kashyap Todi, Luis Leiva, Gilles Bailly, and Antti Oulasvirta. 2021. Adapting User Interfaces with Model-based Reinforcement Learning. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. In press.
- [36] Emanuel Todorov and Michael I Jordan. 2002. Optimal feedback control as a theory of motor coordination. *Nature neuroscience* 5, 11 (2002), 1226–1235.
- [37] Leonard J Trejo, Roman Rosipal, and Bryan Matthews. 2006. Brain-computer interfaces for 1-D and 2-D cursor control: designs using volitional control of the EEG spectrum or steady-state visual evoked potentials. *IEEE transactions on neural systems and rehabilitation engineering* 14, 2 (2006), 225–229.
- [38] Yoji Uno, Mitsuo Kawato, and Rika Suzuki. 1989. Formation and control of optimal trajectory in human multijoint arm movement. *Biological cybernetics* 61, 2 (1989), 89–101.
- [39] Stephan Weibelzahl, Alexandros Paramythi, and Judith Masthoff. 2020. Evaluation of adaptive systems. In *Proceedings of the 28th ACM Conference on User Modeling, Adaptation and personalization*. 394–395.
- [40] Jonathan R Wolpaw, Dennis J McFarland, Gregory W Neat, and Catherine A Forneris. 1991. An EEG-based brain-computer interface for cursor control. *Electroencephalography and clinical neurophysiology* 78, 3 (1991), 252–259.
- [41] Kaiqing Zhang, Zhuoran Yang, and Tamer Başar. 2019. Multi-agent reinforcement learning: A selective overview of theories and algorithms. *arXiv preprint arXiv:1911.10635* (2019).

A UPDATE EQUATIONS

The update equations used by assistant, user, and critic networks are:

$$\theta_{u,a} \leftarrow \theta_{u,a} + \lambda_{u,a} \nabla_{\theta} L^{CLIP}(\theta_{u,a})$$

$$\phi \leftarrow \phi + \lambda_c \nabla_{\phi} \hat{\mathbb{E}}_t \left[(V\phi(s_t) - V^{\pi_{old}}(s_t))^2 \right]$$

Here, $\theta_{u,a}$ and ϕ are the parameters for the user, assistant, and critic function, respectively. $\lambda_{u,a}$ and λ_c are the learning rates for the user, assistant, and critic, respectively. $L^{CLIP}(\theta)$ is the loss function for the actor, and $V_{\phi}(s_t)$ is the value function of the current policy at time step t .

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t [\min(r_t(\theta) \cdot A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \cdot A_t)]$$

where A_t is the advantage at time step t , ϵ is a hyperparameter that controls the amount of "clipping" that occurs, and $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ is the likelihood ratio between the current and old policies at time step t .

$$A^{\pi_{old}}(s_t, a_t) = \sum_{t'=t}^T \gamma^{t'-t} r(s_{t'}, a_{t'}) - V^{\pi_{old}}(s_t)$$

Here, γ is the discount factor, T is the final time step, $r(s_t, a_t)$ is the reward at time step t , and $V^{\pi_{old}}(s_t)$ is the value function of the old policy at time step t .

These updates are similarly described for the one agent setting in [31]

B NETWORK ARCHITECTURE

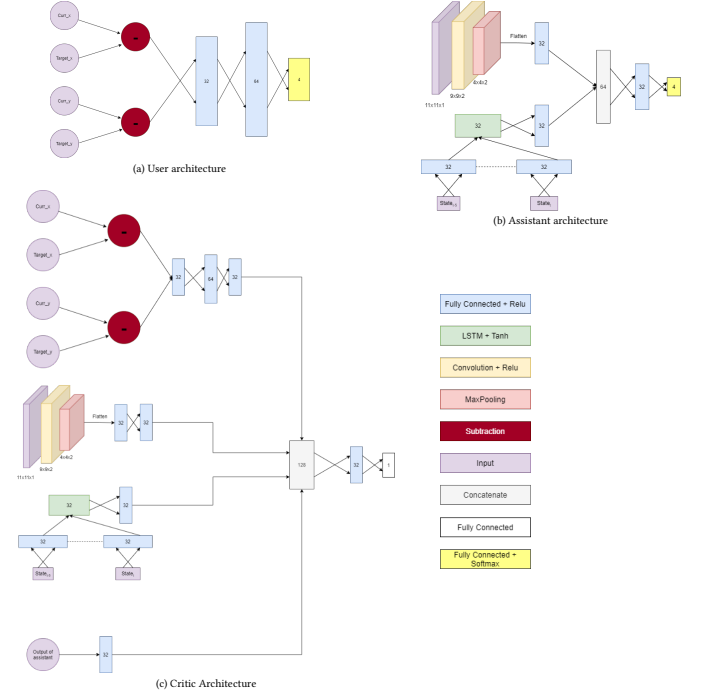


Figure 4: Architectures of the actors and the critics. (a) user architecture, with simple forward neural networks. (b) assistant architecture, which features a convolutional network to deal with grid information, as well as a network with memory, where the state at time t consists of user location (x_t, y_t) and user action (a_t) . (c) Critic architecture.